

Module 5: Software Assurance and Secure SQL

Author: Tara K. Jain (kjain4@jhu.edu)

This report documents the software assurance and SQL injection defenses implemented in the Flask and PostgreSQL Grad Cafe analysis application. It explains reproducible installation, safe SQL composition, query limits, database privileges, dependency analysis, packaging, security scanning, and continuous integration. The supplied screenshots and Snyk Code output provide evidence for the reported results.

Report organization

Section	Page
Step 0: Environment, database setup, and application execution	2
Step 1: Pylint compliance	3
Step 2: SQL injection defenses and LIMIT enforcement	4
Step 3: Database hardening and least privilege	5
Step 4: Python dependency graph	6
Step 5: Reproducible installation and packaging	7
Step 6: Snyk dependency scan	8
Step 6: Snyk Code analysis (extra credit)	9
Step 7: GitHub Actions CI enforcement	10

Project and evidence

Repository SSH URL: `git@github.com:twinzies/jhu_software_concepts.git`

The report uses the answers in *local_files/report.txt*, the four supplied screenshots, *local_files/snyk-code-test.txt*, and the project's source files, configuration, and dependency graph. Results describe the supplied evidence; installation instructions are reproducible procedures rather than a claim that a new clean-environment run was performed for this report.

Step 0: Environment and Application Setup

Module 5 continues the Flask analysis application from Module 4. Use Python 3.10 or later, PostgreSQL, and Graphviz; the CI workflow uses Python 3.12 and PostgreSQL 16. Firefox is also required for Selenium when using the Pull Data feature. The pip and uv installation paths are provided in Step 5.

Environment variables

Run commands from `module_5/` after activating the chosen virtual environment. Copy the example file, replace its placeholder password with the application role's password, and export the settings before launching the app or its subprocesses.

```
cp .env.example .env
# Edit .env and replace the placeholder password before continuing.
set -a
source .env
set +a
```

The required variables are `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, and `DB_PASSWORD`. The example uses localhost, port 5432, database `module_4`, and role `gradcafe_app`. The existing database name is retained even though the project folder is `module_5`. The `.env` file is excluded by `.gitignore`.

One-time database initialization

As the database owner, create the database if it does not already exist, initialize the table, and configure the restricted application role. These administrative commands are separate from routine application execution.

```
createdb module_4
psql -d module_4 -f db/schema.sql
psql -d module_4 -v app_password="$DB_PASSWORD" \
-f db/least_privilege.sql
```

Run the application and analysis

```
python src/load_data.py
python src/query_data.py
python src/orm_queries.py
flask --app app run
```

The loader reads the supplied JSON and appends records not already stored. The query scripts print the eleven analysis results; the Flask page is available at `http://127.0.0.1:5000/analysis`. Pull Data scrapes and loads new records, while Update Analysis refreshes the displayed results.

For verification, run `python -m pytest`. Database tests require access to a dedicated test database through `TEST_DATABASE_URL` or the local PostgreSQL defaults; they may be skipped if PostgreSQL is unavailable. CI supplies its own PostgreSQL service (Step 7).

Step 1: Pylint Compliance

Result

Check	Recorded result
Scope	Python files under module_5/src
Final Pylint rating	10.00/10
Final displayed diagnostics	No warnings or errors shown

Explanation

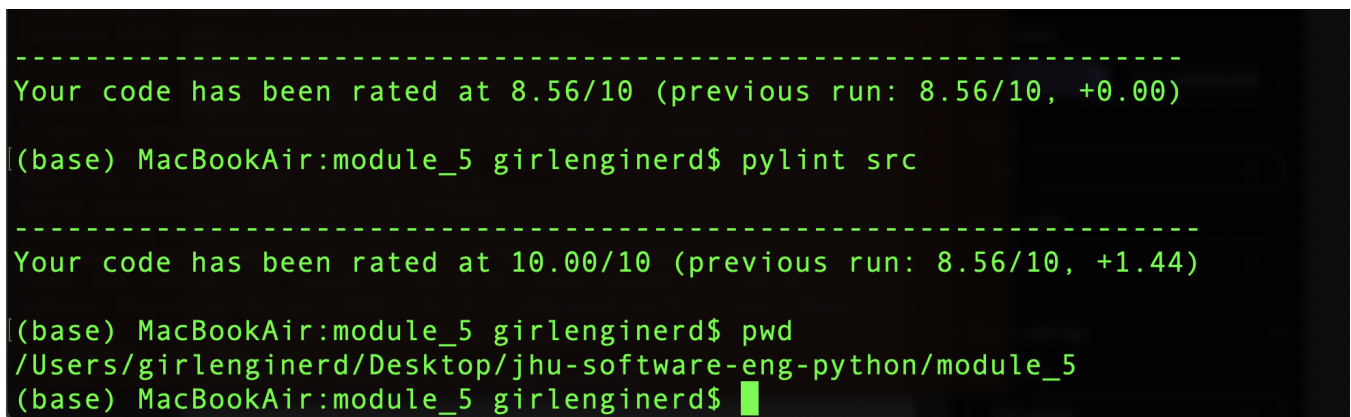
Pylint checks the application source for coding errors, style issues, and maintainability problems. The final recorded run reaches the required score of 10.00/10. Figure 1 also contains an earlier score of 8.56/10; the later result is the final score. The README records the local command, and CI enforces the same source scope with a minimum score of 10.

Commands

```
# Local command recorded in the screenshot and README
pylint src

# CI enforcement
python -m pylint src --fail-under=10
```

Evidence



```
-----
Your code has been rated at 8.56/10 (previous run: 8.56/10, +0.00)

(base) MacBookAir:module_5 girlenginerd$ pylint src

-----
Your code has been rated at 10.00/10 (previous run: 8.56/10, +1.44)

(base) MacBookAir:module_5 girlenginerd$ pwd
/Users/girlenginerd/Desktop/jhu-software-eng-python/module_5
(base) MacBookAir:module_5 girlenginerd$
```

Figure 1. Supplied Pylint screenshot showing the final 10.00/10 score for src and the module_5 working directory.

Step 2: SQL Injection Defenses

What changed and why it is safe

The psycopg query path builds statements with `sql.SQL` and quotes variable table and column names with `sql.Identifier`. Filter values and limits remain placeholders and are supplied separately to `cursor.execute(statement, params)`. The loader follows the same separation for inserted values through `executemany`. User data therefore remains data instead of becoming executable SQL syntax.

The `.format()` calls in this implementation are methods of psycopg `sql.SQL` objects that insert composable SQL fragments. They do not interpolate user values into ordinary Python strings. The alternate SQLAlchemy analysis path also binds values and applies a clamped `.limit(limit)`.

Representative construction and execution

The following condensed example follows the implementation in `query_data.py` and `sql_safety.py`.

```
from psycopg import sql
from sql_safety import clamp_limit

statement = sql.SQL("""
    SELECT COUNT(*) AS fall_2026_count
    FROM {table}
    WHERE LOWER(term) = %(fall_2026)s
    LIMIT {limit}
""").format(
    table=sql.Identifier("applicants"),
    limit=sql.Placeholder("limit"),
)
params = {"fall_2026": "fall 2026", "limit": clamp_limit(limit)}
cursor.execute(statement, params)
```

LIMIT enforcement

`clamp_limit` converts the request to an integer, clamps it to 1-100, and defaults invalid values to 100. The eleven analysis queries and applicant-count query bind the validated limit. The loader reads existing records in limited pages and its advisory-lock `SELECT` uses `LIMIT 1`. `INSERT` statements write bound rows and do not use a `SELECT`-style `LIMIT`. Aggregate limits constrain returned rows, not the number of source records used to compute an aggregate.

Malicious-input checks

`tests/test_sql_injection.py` checks oversized and negative limits, nonnumeric input, `DROP TABLE` payloads, tautologies, and `UNION` payloads. It checks that malicious limit requests return HTTP 200 with a validated limit, injected filter text matches no records, and malicious inserted text is stored literally without dropping the table. These tests are part of the successful CI run shown in Step 7.

Step 3: Database Hardening

Permissions granted and rationale

The app connects as `gradcafe_app`, a login role without superuser, database-creation, role-creation, replication, or row-security-bypass privileges. `CONNECT` permits access to the database, and `USAGE` permits access to the public schema. `SELECT` on applicants supports analysis; `INSERT` allows Pull Data to append newly scraped rows.

The application is not read-only because it adds records, but it does not need to update or delete existing rows. It therefore receives no `UPDATE`, `DELETE`, or `TRUNCATE` privileges and does not own the table. Without ownership or elevated privileges, it cannot `DROP` or `ALTER` that table. This reduces the consequences of compromised application credentials, although the permitted reads and inserts would still be exposed.

SQL used

The following excerpt from `db/least_privilege.sql` is run once by the database owner. The password is supplied as a `psql` variable.

```
CREATE ROLE gradcafe_app LOGIN PASSWORD :'app_password'
    NOSUPERUSER NOCREATEDB NOCREATEROLE
    NOREPLICATION NOBYPASSRLS;
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
SELECT format('GRANT CONNECT ON DATABASE %I TO gradcafe_app',
    current_database()) \gexec
GRANT USAGE ON SCHEMA public TO gradcafe_app;
GRANT SELECT, INSERT ON TABLE applicants TO gradcafe_app;
```

Credentials and privilege evidence

`src/db.py` reads connection values from the five `DB_*` environment variables documented in Step 0. Real credentials are not embedded in the application source; `.env.example` supplies placeholders and `.gitignore` excludes `.env`.

```
(.venv) (base) MacBookAir:module_5 girlenginerd$ psql -d module_4 -c '\du gradcafe_app' -c '\dp applicants'
```

List of roles		
Role name	Attributes	Member of
gradcafe_app		{}

			Access privileges		Column privileges	Policies
Schema	Name	Type	Access privileges		Column privileges	Policies
public	applicants	table	girlenginerd=arwdDxt/girlenginerd+			
			gradcafe_app=ar/girlenginerd			

(1 row)

Figure 2. PostgreSQL role and table privileges. The `gradcafe_app=ar` entry grants `INSERT` (a) and `SELECT` (r); the role has no elevated attributes or role memberships shown.

Step 4: Python Dependency Graph

Dependency summary

The dependency graph shows the relationships between the project's Python modules and the external libraries imported by the application. Each arrow points from a dependency to the module that uses it. The main module, `app.py`, uses `Flask` to provide the web interface and `query_data` to retrieve analysis results. The `query_data` module uses `db` for PostgreSQL connections and `Psycopyg` for database queries. Both `app.py` and `query_data` use `sql_safety` for shared limit validation, while `query_data` also uses its quoted table identifier. Most of the graph's visual complexity comes from `Flask` and `Psycopyg` internals, while the project modules separate the interface, analysis, connection configuration, and SQL safety responsibilities.

Generation

With `pydeps` installed in the Python environment and `Graphviz's dot` executable available, the application-focused graph can be generated from `module_5/` as follows. The supplied `dependency.svg` is reproduced below.

```
cd src
pydeps app.py --noshw -T svg -o ../dependency.svg
cd ..
```

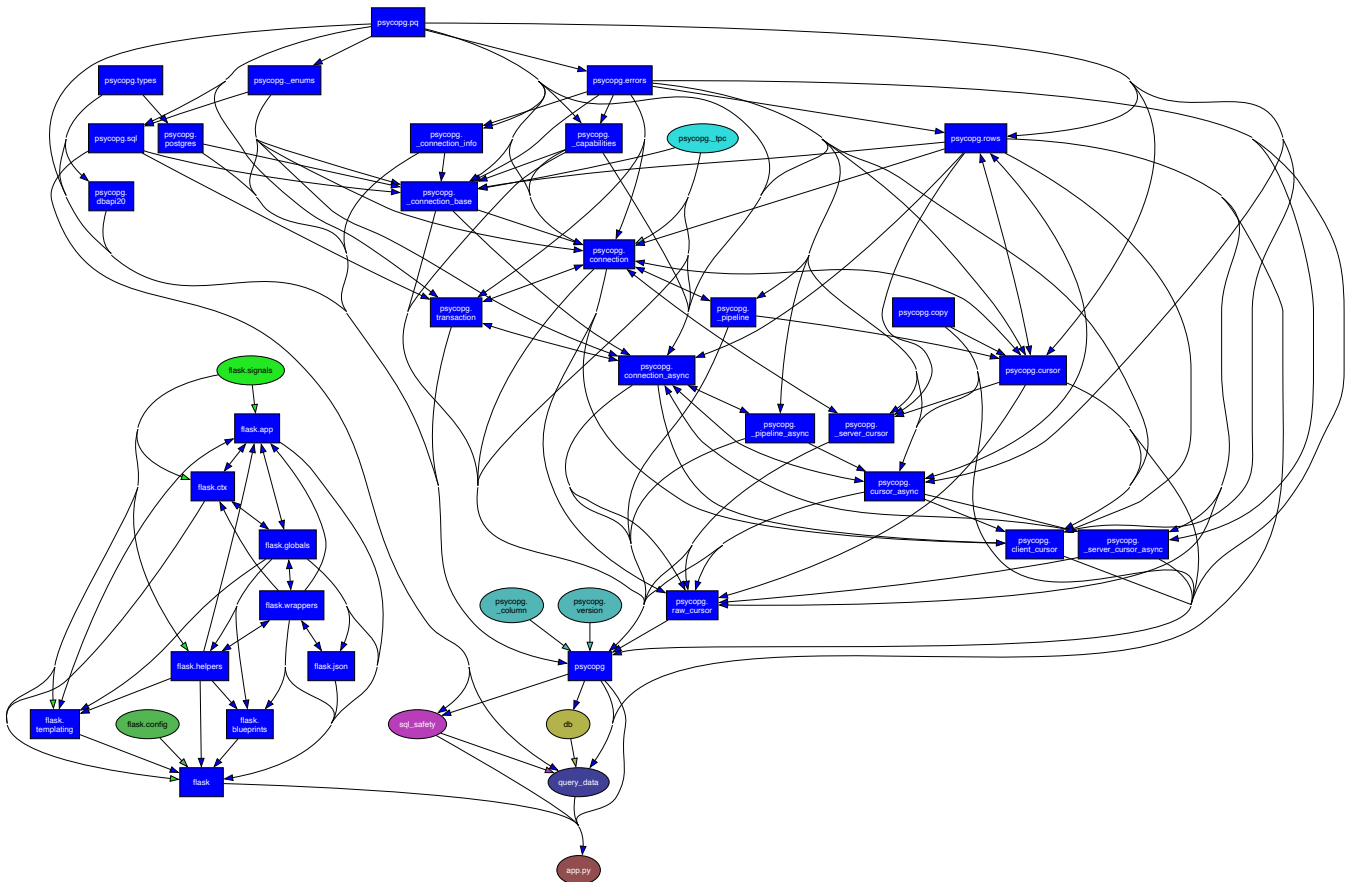


Figure 3. The supplied `dependency.svg`, retained as vector artwork for zooming. CI generates a broader, filtered module graph and verifies that its SVG output is nonempty.

Step 5: Reproducible Installation and Packaging

Fresh installation with pip and venv

Choose one of the following alternatives from module_5/. Both install the runtime dependencies, development tools, and local project. PostgreSQL and Graphviz must also be available on the host.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -e .
```

Alternate installation with uv

With the uv CLI installed, create and activate a separate environment. If switching from the pip environment above, deactivate it first.

```
uv venv .venv-uv
source .venv-uv/bin/activate
uv pip install -r requirements.txt
uv pip install -e .
```

After either path, configure and export the `DB_*` variables, initialize the database as needed, and run the application using the commands in Step 0. The same Flask, query, lint, and test commands apply to both environments.

Requirements and tools

requirements.txt includes Flask, Psycopg, SQLAlchemy, BeautifulSoup, Selenium, and urllib3 for runtime use. It also includes pytest and pytest-cov, the required pylint and pydeps tools, and Sphinx documentation dependencies. These requirements support installation from a new environment; because versions are expressed as ranges, they do not guarantee an identical dependency set across installation dates.

Why packaging matters

Packaging makes the project installable so Python can find its modules consistently during local development, testing, and CI when the package is installed. The setup.py file declares the project name, Python requirement, source modules, and dependencies. An editable installation, `pip install -e .`, points to the working source files, so ordinary code changes do not require reinstalling. This reduces dependence on a particular working directory or manual Python import-path changes.

The package is named gradcafe-analysis, version 5.0.0, and discovers its Python modules under src/. The supplied CI workflow installs requirements and the test configuration adds src to the import path; the fresh-install procedures above additionally install the project itself.

Step 6: Snyc Dependency Scan

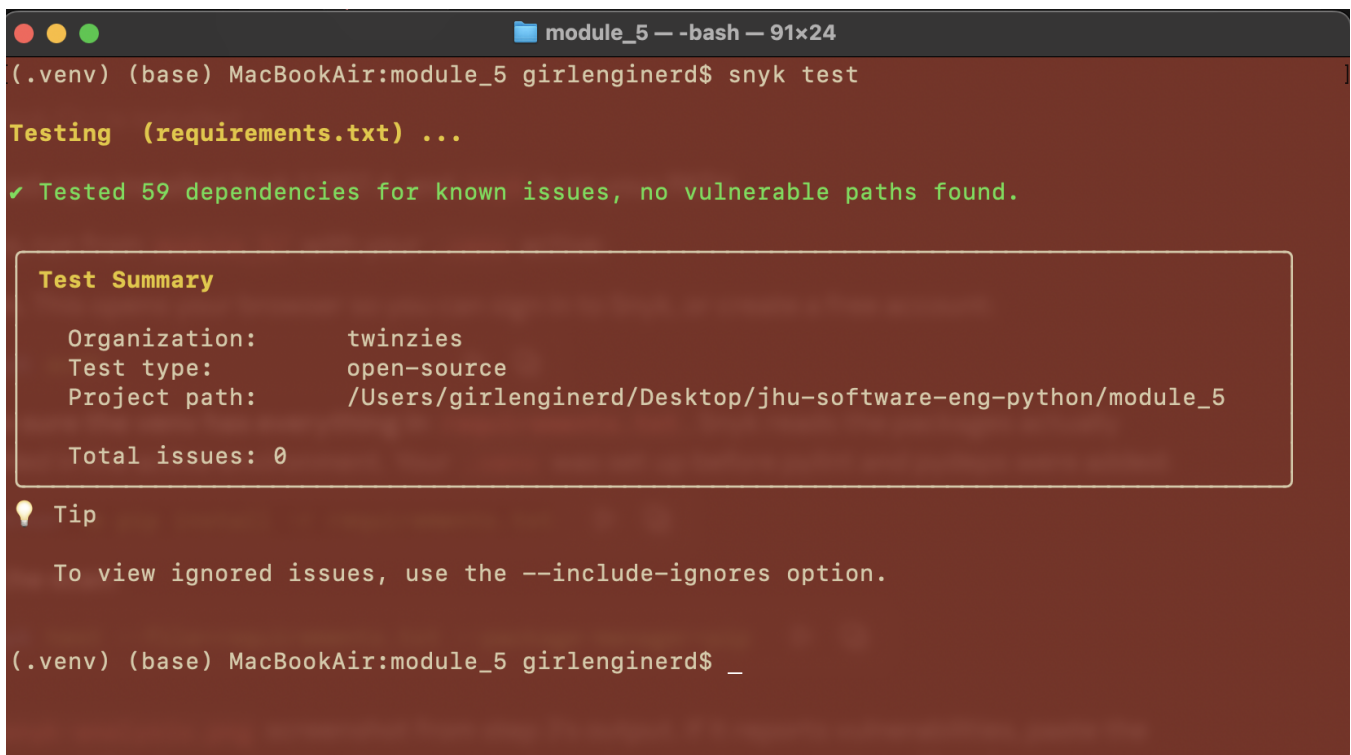
Result

Measure	Recorded result
Command	snyk test
Manifest	requirements.txt
Dependencies tested	59
Total issues	0
Vulnerable paths	None found

Explanation

The open-source dependency scan checks the installed dependency tree against known vulnerabilities. The supplied result reports 59 dependencies tested and no vulnerable paths, so no dependency patch or removal was indicated by this scan. This result describes the dependency scan at the time it was run; it does not establish that the application source is free of security issues. Source-code analysis is reported separately on the following page.

Evidence



```
module_5 — -bash — 91x24
(.venv) (base) MacBookAir:module_5 girlenginerd$ snyk test

Testing (requirements.txt) ...

✓ Tested 59 dependencies for known issues, no vulnerable paths found.

Test Summary
Organization:    twinzies
Test type:      open-source
Project path:    /Users/girlenginerd/Desktop/jhu-software-eng-python/module_5
Total issues: 0

💡 Tip
To view ignored issues, use the --include-ignores option.

(.venv) (base) MacBookAir:module_5 girlenginerd$ _
```

Figure 4. Supplied snyk-analysis.png showing the successful dependency scan and zero reported issues.

Step 6: Snyk Code Analysis (Extra Credit)

Result and scope

The supplied output from snyk code test reports 115 potential issues: 2 high, 42 medium, and 71 low, with none ignored. Most findings occur in installed libraries under .venv or saved source history under .history. Those local paths are distinct from the application source; excluding them from a repository does not resolve a finding in application code.

Recorded output excerpts

The following excerpts are transcribed from local_files/snyk-code-test.txt; line references describe the scanned files at the time of the run.

```
Test type: Static code analysis
Total issues: 115
Ignored issues: 0 [0 HIGH  0 MEDIUM  0 LOW]
Open issues: 115 [2 HIGH  42 MEDIUM  71 LOW]

[LOW] Path Traversal
Path: src/load_data.py, line 118

[MEDIUM] Server Information Exposure
Path: src/app.py, line 168
```

Application findings and proposed remediation

Path traversal: Snyk reports that a command-line argument reaches pathlib.Path and may allow arbitrary file reads. The loader intentionally accepts a JSON input path, so exploitability depends on who controls that argument and the process permissions. A mitigation is to resolve the path and restrict it to an approved input directory when the caller is untrusted.

Server information exposure: Snyk reports that exception information can reach the analysis response. A mitigation is to show a generic error message to the user and retain detailed exceptions only in server-side logs.

Remediation status

These are static-analysis findings that require review; they are not proof of successful exploitation. The supplied evidence does not document completed fixes or a clean Snyk Code rescan. The two application findings therefore remain open for review. Findings within installed libraries and historical files should be assessed separately, with scan scope narrowed to maintained source when evaluating application changes.

Evidence source: local_files/snyk-code-test.txt, including the summary and the src/load_data.py and src/app.py entries.

Step 7: GitHub Actions CI Enforcement

The repository-level `.github/workflows/ci.yml` defines four separate jobs and runs on push, pull_request, and workflow_dispatch. Commands use `module_5` as their default working directory. This brings linting, dependency analysis, vulnerability scanning, and tests into the development workflow.

Job	Enforcement
Pylint	Runs <code>python -m pylint src --fail-under=10</code> .
Dependency graph	Installs Graphviz; runs <code>pydeps</code> ; requires <code>test -s dependency.svg</code> and uploads the graph artifact.
Snyk dependency scan	Runs <code>snyk test</code> against <code>requirements.txt</code> with <code>--severity-threshold=high</code> ; uses the <code>SNYK_TOKEN</code> repository secret.
Pytest	Uses PostgreSQL 16 and a dedicated test database; runs <code>python -m pytest</code> , failing on test failures or coverage below 100% as configured in <code>pytest.ini</code> .

Successful run evidence

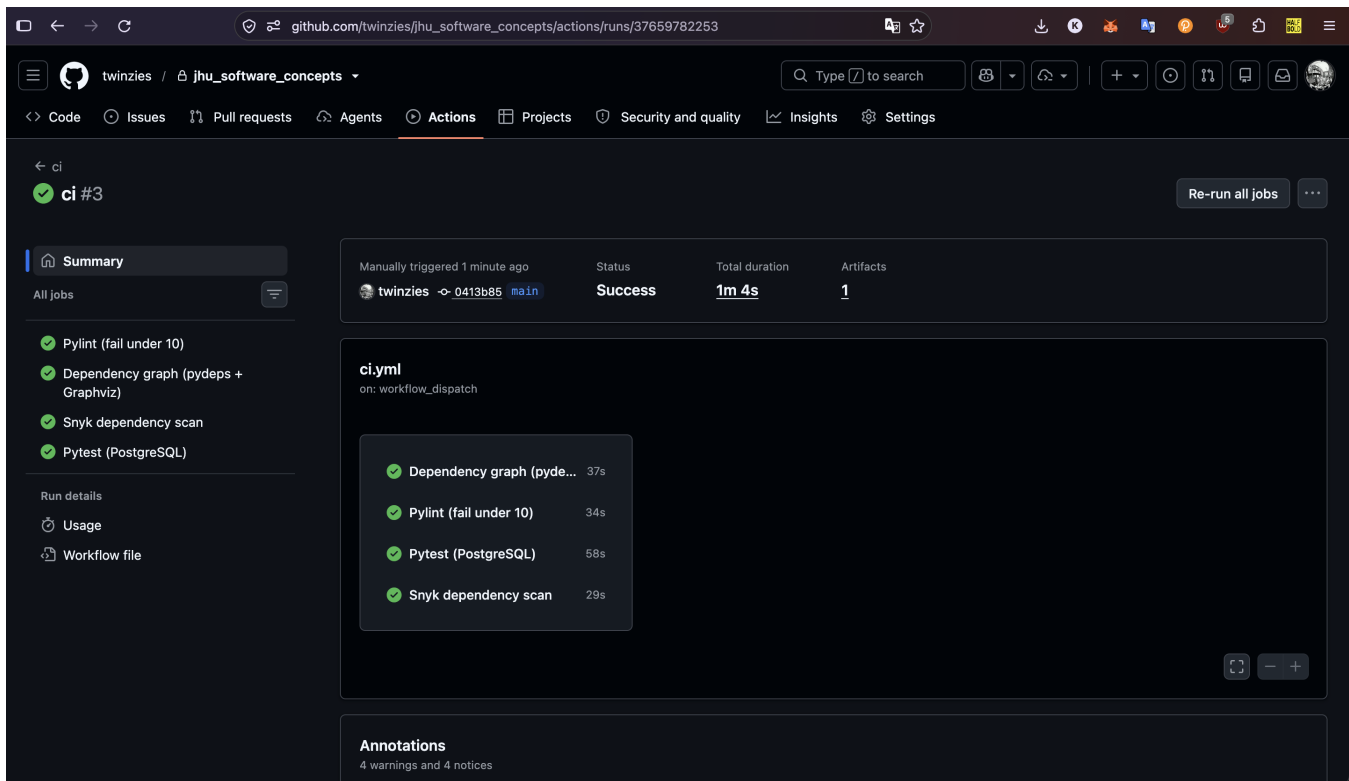


Figure 5. Supplied GitHub Actions screenshot: ci #3, commit 0413b85 on main, status Success, all four jobs green, and one generated artifact. This recorded run was manually triggered and completed in 1 minute 4 seconds.

The screenshot demonstrates a successful run; the workflow file supplies the push and pull-request trigger configuration. Its annotation area also reports four warnings and four notices, so the evidence supports job success without implying that every workflow annotation is absent.